

Callables and First-Class Objects

Functions and methods aren't the only things that you can call in Python. Any object that implements the *callable operator*—the two parentheses `()`—is a *callable* object. For example, if you have a `def hello():` statement, you can think of the code as a variable named `hello` that contains a function object. Using the callable operator on this variable calls the function in the variable: `hello()`.

Classes are an OOP concept, and a class is an example of a callable object that isn't a function or method. For example, the `date` class in the `datetime` module is called using the callable operator, as in the code `datetime.date(2020, 1, 1)`. When the class object is called, the code inside the class's `__init__()` method is run. Chapter 15 has more details about classes.

Functions are *first-class objects* in Python, meaning you can store them in variables, pass them as arguments in function calls, return them from function calls, and do anything else you can do with an object. Think of a `def` statement as assigning a function object to a variable. For example, you could create a `spam()` function that you can then call:

```
>>> def spam():
...     print('Spam! Spam! Spam!')
...
>>> spam()
Spam! Spam! Spam!
```

You can also assign the `spam()` function object to other variables. When you call the variable you've assigned the function object to, Python executes the function:

```
>>> eggs = spam
>>> eggs()
Spam! Spam! Spam!
```

These are called *aliases*, which are different names for existing functions. They're often used if you need to rename a function. But a large amount of existing code uses the old name, and it would be too much work to change it.